

config_endpoint

REFERENCE

API consumer

add, update, or delete an MQTT endpoint configuration on the reader

Overview

`config_endpoint` provisions the reader's MQTT endpoints — it **adds**, **updates**, or **deletes** an endpoint configuration (chosen by `epConfig.operation`). One payload defines an endpoint's type, protocol, broker `url` / `port`, TLS `verificationType` and certificates, MQTT `keepAlive` /session/reconnect settings, publish/subscribe topics, optional broker credentials, and per-endpoint event routing. It is a state-changing command on the device-management (MGMT) endpoint; the response is **status-only**. Read the result back with `get_endpoint_config`.

`config_endpoint` is **state-changing and can carry broker credentials** — and reconfiguring the endpoint that carries your own control session can sever it. See the session-sever callout in [Notes and Caveats](#) and the credential/TLS handling in [Request Semantics](#). A saved endpoint is reversible with `operation: delete`.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (writes/removes a saved endpoint; may activate it)
Reversibility	REVERSIBLE (remove with <code>operation: delete</code> ; change with <code>operation: update</code>)
Supported Operations	<code>add</code> (new), <code>update</code> (existing), <code>delete</code> (existing)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>epConfig</code> (with <code>operation</code> and <code>configuration</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>V1.1</code> (API Version Reporting)
Related	<code>get_endpoint_config</code> , <code>config_events</code> , <code>install_certificate</code> , <code>reboot</code>

When to Use

Use `config_endpoint` to **provision or maintain the reader's MQTT wiring** after the initial `MDM` endpoint exists (that first endpoint is set up with 123RFID Desktop; all others are added remotely with `config_endpoint`).

- `add` — create a new endpoint (`MGMT`, `CTRL`, `DATA1`, ...). The `endpointName` must not already exist (duplicate → code `10`).
- `update` — modify an existing endpoint (e.g. change `url`, or flip `activate`). The `endpointName` must already exist.

- **delete** — remove an existing endpoint (the contract's example sends only **endpointName** + **epType**).
- **Pre-provision safely** — send **activate: false** to save an endpoint without activating it, then activate later with **update** .

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<tenantId>/MGMT/clients/cmnd/<serial>	On the management endpoint.
Response	Reader → App	<tenantId>/MGMT/clients/resp/<serial>	Correlated reply; the in-payload requestId links the twin topics.

Topics follow the pattern **<tenantId>/<topic>/<serial>** — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example **RFD40-212735201D0053**). **App → Reader** means your app publishes and the reader subscribes; **Reader → App** means the reader publishes and your app subscribes.

The **topic** values you configure inside **mqttParams** are **base** topics (e.g. **MGMT/clients/resp**). The reader constructs the full topic at runtime as **<tenantId>/<topic>/<serial>** — never include the tenant or serial in **topic** .

Credentials are presented once at CONNECT (and, for a configured endpoint, via **mqttParams**). The **requestId** you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- **epConfig.operation** (request) — **add** / **update** / **delete** ; determines the action.
- **configuration.endpointName** (request) — the endpoint's identity (create target / update / delete key).
- **configuration.epType** (request) — the endpoint's role (**MGMT** , **CTRL** , **DATA1** , ...).
- **configuration.activate** (request) — **true** activates immediately; **false** saves without activating.
- **configuration.securityParams** (request) — for **MQTT_TLS** ; the referenced certificates must be pre-installed via [install_certificate](#) .
- **response.code** (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The envelope is **command** , **requestId** , and the **epConfig** payload (**operation** + **configuration**). Pick an **operation / endpoint** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Operation / endpoint

Add MGMT endpoint for MQTT TLS ▾

Request payload

Example: add, MGMT, MQTT_TLS

```
1      {
2        "command": "config_endpoint" ,
3        "requestId": "1233" ,
4        "epConfig": {
5          "operation": "add" ,
6          "configuration": {
7            "endpointName": "mgmt_tls" ,
8            "epType": "MGMT" ,
9            "protocol": "MQTT_TLS" ,
10           "activate": true ,
11           "url": "broker.ip.com" ,
12           "port": 8883 ,
13           "verificationType": "VERIFY_HOST_AND_PEER" ,
14           "qosCommon": 1 ,
15           "tenantId": "ZEBRA" ,
16           "mqttParams": {
17             "keepAlive": 300 ,
18             "cleanSession": true ,
19             "reconnectDelayMin": 5 ,
20             "reconnectDelayMax": 60 ,
21             "publishTopics": [
22               {
23                 "topic": "MGMT/clients/resp" ,
24                 "qos": 1 ,
25                 "retain": false
26               },
27               {
28                 "topic": "MGMT/clients/event" ,
29                 "qos": 1 ,
30                 "retain": false
31               },
32               {
33                 "topic": "MGMT/clients/rfid" ,
34                 "qos": 0 ,
35                 "retain": true
36               }
37             ],
38             "subscribeTopics": [
39               {
40                 "topic": "MGMT/clients/cmnd" ,
41                 "qos": 0
42               }
43             ]
44           },
45           "securityParams": {
46             "format": "PEM" ,
47             "caCertificateFile": "mqtt_ca_cert" ,
48             "clientCert": "mqtt_client_cert" ,
49             "clientKey": "mqtt_client_key"
50           }
51         }
52       }
53     }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Fixed comma
2	requestId *	string	—	abc123	A unique ide echoed back
3	epConfig *	object	—	—	The endpoint payload. Req which also re and configu
4	operation *	enum	add delete update	add	add a new er existing one, Required with Request Sem
5	configura tion *	object	—	—	The endpoint Required with contract requ epType , pro url , verifi port for eve its own dele examples see Request Sem
6	endpoi ntName *	string	—	mgmt	The endpoint add/update/c
7	epType *	enum	MGMT MGMT_EVT CTRL DATA1 DATA2 SOTI MDM	MGMT	The endpoint is currently n
8	protoc ol *	enum	MQTT MQTT_TLS	MQTT	MQTT (plain, t MQTT_TLS (T 8883). Both l unchanged v get_endpoi Request Sem
9	activa te *	boolean	—	true	true activate immediately; without activ with update)
10	url *	string	—	soti.example.com	Broker addre

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	verifi cation * Type	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_HOST_AND_PEER	TLS verificat Use VERIFY_ VERIFY_NON production, a VERIFY_NON
12	port *	integer	—	1883	Broker port (‘ TLS).
13	qosComm on	integer	—	1	Default QoS l
14	tenantI d	string	—	ZEBRA	Tenant identi of every on-v of the contra but send it: w resolves. Bol characters; a returns code
15	mqttPar ams	object	—	—	MQTT connec topics, and (c credentials.
16	keepA live	integer	max 65535	300	Keep-alive in
17	clean Sessi on	boolean	—	true	Whether to s session.
18	recon nectD elayM in	integer	—	5	Minimum rec
19	recon nectD elayM ax	integer	—	60	Maximum rec
20	clien tId	string	—	zebra-rfd40-01	MQTT client connection.
21	usern ame	string	—	mqttuser	Broker usern requires user Redact it in l
22	passw ord	string	—	—	Broker passw requires pass — a secret; n

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	publi shTop ics	array	—	—	Base topics t publishes to (exceeding re Entries are p examples or responses, [connection e Will. On DAT/ guidance pla at [1] — see Each item rec retain .
24	t o p * i c	string	—	zebra/rfd40/01/data	Base topic or wire to <tenantId> Do not includ
25	q o * s	integer	—	1	QoS for this p
26	r e t * a i n	boolean	—	false	Whether the message.
27	subsc ribeT opics	array	—	—	Base topics t subscribes to (exceeding re item requires
28	t o p * i c	string	—	zebra/rfd40/01/data	Base topic or wire to <tenantId>
29	q o * s	integer	—	0	QoS for this s
30	r e t * a i n	boolean	—	—	Retain flag fc

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
31	securityParams	object	—	—	TLS parameter. Required with format . The certificates not installed via install_certificateFile .
32	format *	enum	PEM PFX	PEM	Certificate format. PEM and PFX , currently supported.
33	certificateFile	string	—	cloud-trusted-ca-certs.crt	Installed CA certificate name (a name).
34	clientCert	string	—	client_cert.pem	Installed client certificate reference name.
35	clientKey	string	—	client_key.pem	Installed client key name.
36	eventConfiguration	object	—	—	Per-endpoint configuration. Same shape as eventConfiguration selecting which endpoint emit heartbeat set are functional Semantics.
37	antenna	boolean	—	false	set to true to enable antenna events
38	terminalConnection	boolean	—	false	set to true to enable connection/disconnection events
39	firmwareUpdate	boolean	—	true	set to true to enable firmware update events
40	gpi	boolean	—	false	set to true to enable GPIO events
41	network	boolean	—	true	set to true to enable network events
42	exceptions	boolean	—	false	set to true to enable exception events
43	ntp	boolean	—	true	set to true to enable NTP alerts

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
44	userApp	boolean	—	false	set to true to events
45	heartbeat	boolean	—	false	set to true to
46	power	boolean	—	true	set to true to alerts
47	battery	boolean	—	true	set to true to
48	temperature	boolean	—	false	set to true to alerts
49	fileDownload	boolean	—	true	set to true to alerts
50	cpuUsage	boolean	—	false	set to true to alerts
51	flashUsage	boolean	—	false	set to true to alerts
52	ramUsage	boolean	—	false	set to true to alerts
53	heartbeatConfiguration	object	—	—	Heartbeat se heartbeat is
54	interval	integer	max 65535	100	Seconds bet most 65,535.
55	inventoryStatus	boolean	—	true	set to true to status as par message

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
56	batteryStatus	boolean	—	true	set to true to as part of he
57	userApps	boolean	—	—	set to true to status as par message
58	cpuThreshold	integer	—	0	cpu usage va alert will be t
59	ramThreshold	integer	—	0	ram usage va alert will be t
60	flashThreshold	integer	—	0	flash usage v alert will be t
61	temperatureThreshold	integer	—	0	temperature which alert w
62	batching	object	—	—	Report batch Optional; wh reader's built and no sourc A report is pu limit is reach
63	reportingInterval	integer	1000–60000	5000	Reporting tim tags, in millis to the minimu
64	maxPayloadSizePerReport	integer	1024–5120	1024	Maximum pa in bytes. 0 fa minimum, 10

Request Semantics

Modes. `epConfig.operation` selects the action. The contract requires the same seven `configuration` fields for every operation — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType` and `port` — but its own `delete` example sends only `endpointName` and `epType`, and its `update` example omits `verificationType` and `port`. A validator built strictly from the contract rejects both examples; this is a known issue in the source schema:

- `add` — creates a new endpoint. Supply the full `configuration` (the 7 fields `add` requires — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType`, `port` — plus optional `mqttParams` / `securityParams` / `eventConfiguration`). Fails with code `10` if the `endpointName` already exists.
- `update` — modifies an existing endpoint (matched by `endpointName`). The contract's example sends `endpointName` and `epType` plus the fields being changed.
- `delete` — removes an existing endpoint. The contract's example sends only `endpointName` and `epType`.

`add` is **not idempotent**. If a response is lost (e.g. a timeout), the endpoint may already have been created — **read back with `get_endpoint_config` before retrying** rather than blindly re-sending, or the retry will be a duplicate `add`.

- **Payloads can carry broker credentials.** `mqttParams.username` / `password` are supplied in the request when the broker requires them — masked in the example on this page. Redact them in logs, tickets, and screenshots, and send `config_endpoint` only over secured channels. (Certificate fields — `caCertificateFile` / `clientCert` / `clientKey` — are **reference names** of already-installed certificates, not key material.)
- **Optional MQTT connection fields.** `mqttParams.clientId` sets the MQTT client ID used for the broker connection; `cleanSession` defaults to `true`.
- **TLS needs pre-installed certificates.** For `protocol: MQTT_TLS`, the `securityParams` `caCertificateFile` / `clientCert` / `clientKey` must reference certificates already installed via `install_certificate`. Only `format: PEM` is supported today. The protocol literal is spelled `MQTT_TLS` in both directions — one enum governs the request and the `get_endpoint_config` read-back — so send and match it exactly.
- **Topic slots are positional.** `publishTopics` accepts **at most 3** entries (exceeding → code `25`) and `subscribeTopics` **at most 1** (exceeding → code `26`); both arrays are optional — the contract's own `DATA1` read-back returns `subscribeTopics: []`. The three publish slots carry command **responses** (`.../clients/resp`), **events** — heartbeat, alerts (`.../clients/event`), then the endpoint's **connection-event** topic (`.../clients/rfid`, which carries `mqttConnEVT`); `subscribeTopics[0]` is **command ingress**, where the host publishes and the reader subscribes (`.../clients/cmdn`). Position is what routes — the reader binds each entry by **index, not by base name** — so keep the configured order, leave omitted indices unused, and match a `get_endpoint_config` read-back by index rather than by topic name. Configure only the base `topic`; the reader wraps it as `<tenantId>/<topic>/<serial>`.
- **DATA endpoints: put the tag-data topic at `publishTopics[1]`.** On `epType` `DATA1` the reader streams tag data from the **second** publish entry (`DATA2` is currently not supported), so configure an unused placeholder at `[0]` and the tag-data topic (e.g. `DATA/clients/data1event`) at `[1]`: `[{ "topic": "DATA/clients/unused", "qos": 0, "retain": false }, { "topic": "DATA/clients/data1event", "qos": 1, "retain": false }]`. A single-entry array is the trap: the one entry lands in the **reply** slot, the `add` still succeeds, and no tag data ever arrives — which is what the contract's one-entry `DATA1` read-back is

showing. A read-back showing `activate: true` only attests that the endpoint connected; it does not prove tag data is flowing.

- `verificationType` : use `VERIFY_HOST_AND_PEER` . It verifies the broker's certificate chain and that the certificate matches the host. `VERIFY_NONE` turns verification off; use it only on an isolated test network, and never on an `MQTT_TLS` endpoint, which rejects it with code `37` . `get_endpoint_config` reads back the value you sent, so a client doing read-modify-write can echo it.
- **Event routing.** `configuration.eventConfiguration` uses the same shape as `config_events` ; only 7 of its toggles are functional today (the rest are reserved). See the Event routing fields table on the `get_endpoint_config` page for the full supported-vs-reserved split.

Topic slots by `epType` . How the positional layout above lands per endpoint type. The contract's examples show the command/response layout; the data-endpoint layout (+) is firmware guidance that the contract does not state:

<code>epType</code>	<code>publishTopics[0]</code>	<code>publishTopics[1]</code>	<code>publishTopics[2]</code>	<code>subscribeTopics[0]</code>
<code>MGMT</code> · <code>MDM</code> · <code>SOTI</code>	command responses (<code>.../resp</code>)	events (<code>.../event</code>)	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>CTRL</code>	command responses (<code>.../resp</code>)	stored, not used for events	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>MGMT_EVT</code>	not used	not used	Last Will	events are currently published here (a firmware defect)
<code>DATA1</code>	unused placeholder †	tag-data stream (<code>.../data1event</code>) †	—	— (does not subscribe)

The command/response endpoints (`MGMT` , `MDM` , `SOTI` , `CTRL`) share the `[0]` and `[2]` layout; on `CTRL` , slot `[1]` is stored but not used for events. A `MGMT_EVT` endpoint currently publishes its events on its subscribe topic instead of `publishTopics[1]` , which the API owners record as a firmware defect; see [Endpoints and Topic Slots](#). Match read-back entries by index, not by base name — the `resp` / `event` / `rfid` names are conventions.

† Tag data binds to `publishTopics[1]` , so a data endpoint needs a throwaway entry at `[0]` to push the real topic into slot `[1]` — see the DATA-endpoint bullet above.

Response Payload

`config_endpoint` returns a **status-only** envelope — `command` , `requestId` , `apiVersion` , and `response` . There is no configuration echo; confirm the change with `get_endpoint_config` .

Response payload

```

1  {
2    "command" : "config_endpoint" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — config_endpoint echoes no configuration.
5	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Response Codes

`config_endpoint` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the endpoint was added, updated, or deleted.

Code	Description
10	Configuration already exists (<code>IOT_ERROR_CONFIG_ALREADY_EXIST</code>) — an <code>add</code> used an <code>endpointName</code> that already exists. Use <code>update</code> , or delete it first.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>operation</code> , <code>epType</code> , <code>protocol</code> , <code>verificationType</code> , <code>format</code>) is outside its allowed set.
25	Max 3 publish topics exceeded (<code>IOT_ERROR_PUBLISH_TOPICS_EXCEEDED</code>) — more than 3 <code>publishTopics</code> .
26	Max 1 subscribe topic exceeded (<code>IOT_ERROR_SUBSCRIBE_TOPIC_EXCEEDED</code>) — more than 1 <code>subscribeTopics</code> .
27	Invalid tenant ID length (<code>IOT_ERROR_INVALID_TENANTID_LENGTH</code>) — <code>tenantId</code> exceeds the allowed length.
33	Endpoint already configured (<code>IOT_ERROR_ENDPOINT_ALREADY_CONFIGURED</code>) — an endpoint with the same configuration already exists.
35	Can't modify an active endpoint (<code>IOT_ERROR_CANT_MODIFY_ACTIVE_ENDPOINT</code>) — the target endpoint is currently active. Deactivate it before modifying it.
37	Invalid verification type for MQTT TLS (<code>IOT_INVALID_MQTT_TLS_VERIFICATION_TYPE</code>) — <code>protocol: MQTT_TLS</code> was sent with <code>verificationType: VERIFY_NONE</code> . Use <code>VERIFY_HOST_AND_PEER</code> .
101	Default error — generic catch-all. See the duplicate- <code>add</code> note below.

Duplicate `add` may return the generic 101. The documented code for adding an `endpointName` that already exists is 10 ("Configuration already exists"), but the firmware has been observed returning the generic 101 ("Default error") for a duplicate `add` instead of 10 . Treat a 101 after a re-sent `add` as a probable duplicate and verify with `get_endpoint_config` .

Examples

Publish a `config_endpoint` request; the device replies with the status-only envelope under [Response Payload](#). Every contract example — each `add` , the `delete` and the partial `update` — is in the **Operation / endpoint** dropdown under [Request Payload](#). The examples below show what the dropdown does not.

A **plain-MQTT DATA1** `add` with broker credentials (`activate: true`), the tag-data topic at publish slot [1] with an unused placeholder at [0] — the layout the firmware guidance under [Request Semantics](#) describes — and no TLS:

Request — add (plain MQTT, with credentials)

```
1  {
2    "command" : "config_endpoint" ,
3    "requestId" : "abc123" ,
4    "epConfig" : {
5      "operation" : "add" ,
6      "configuration" : {
7        "endpointName" : "dataEP" ,
8        "epType" : "DATA1" ,
9        "protocol" : "MQTT" ,
10       "activate" : true ,
11       "url" : "broker.example.com" ,
12       "verificationType" : "VERIFY_NONE" ,
13       "port" : 1883 ,
14       "qosCommon" : 1 ,
15       "tenantId" : "zebra" ,
16       "mqttParams" : {
17         "keepAlive" : 300 ,
18         "cleanSession" : true ,
19         "reconnectDelayMin" : 50 ,
20         "reconnectDelayMax" : 500 ,
21         "username" : "*****" ,
22         "password" : "XXXXXXXX" ,
23         "publishTopics" : [
24           {
25             "topic" : "DATA/clients/unused" ,
26             "qos" : 0 ,
27             "retain" : false
28           } ,
29           {
30             "topic" : "DATA/clients/data1event" ,
31             "qos" : 1 ,
32             "retain" : false
33           }
34         ]
35       }
36     }
37   }
38 }
```

A **duplicate** **add** returns code **10** (or, on some firmware, the generic **101** — see the note above):

Response — duplicate add (code 10)

```
1  {
2    "command" : "config_endpoint" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 10 ,
7      "description" : "Configuration already exists"
8    }
9  }
```

Notes and Caveats

⚠ RECONFIGURING THE ACTIVE CONTROL ENDPOINT CAN SEVER YOUR SESSION

Activating a *new* endpoint of a different role coexists with the endpoint carrying your control session (e.g. activating a `DATA1` endpoint does not drop the `MDM` / `MGMT` control channel). But changing the broker `url` / `port` / `protocol` of — or deleting — the endpoint that carries your **own** control session can sever it. When in doubt, save with `activate: false` and switch deliberately.

Because the response is status-only, **verify the result with** `get_endpoint_config` — confirm the endpoint appears in `savedEndpoints.epNames[]` and (if `activate: true`) under `activeEndpoints`. For the concept-level view, see Endpoints, Topics, and Channels.

Related: [Endpoints, Topics, and Channels](#) · [get_endpoint_config](#) · [config_events](#) · [install_certificate](#) · [Response Codes](#)